

Evaluating Optimization Algorithms on Flow Networks

Ben Heinze

May 7, 2026

1 Introduction

Jake is a civil engineer fresh out of college who has been assigned to help the great city of Bozeman reduce its terrible traffic. A large proportion of Gallatin County residents commute from Belgrade to Montana State University for work and school, causing severe congestion during the academic year. Jake's boss finally decides to invest money into improving the road system, but the city cannot afford to expand every road. Instead, they must determine which roads are the most important to improve. Fortunately, Jake took a graduate Algorithms course at MSU. He models the road system as a flow network. In this network, Belgrade is the source s , Montana State University is the sink t , and each road has a capacity, representing the maximum number of cars that can travel on that road without causing congestion. The maximum flow problem asks: What is the greatest number of cars that can travel from Belgrade to MSU at the same time without exceeding the capacity of any road? By solving this problem, Jake can identify which roads create bottlenecks and determine where increasing capacity would most effectively reduce traffic.

This is one of many real-world applications of flow networks: airline passenger optimization, business supply-chain optimization, and even image segmentation in computer vision. Three of the most common techniques used to solve these problems include Ford-Fulkerson's depth-first search algorithm, the Ford-Fulkerson's breadth-first search algorithm called Edmonds-Karp, and utilizing linear programming. Although they all find optimal solutions to the max flow problem, this paper performs a side-by-side analysis of three flow network algorithms to determine which one tends to perform the fastest given graphs of increasing node sizes and densities. Which algorithm will consistently perform the best, the Ford-Fulkerson algorithm, the Edmonds-Karp algorithm, or the CPLEX linear programming solver?

2 Flow Networks

Flow networks consist of four attributes: the graphs, capacity, source, and sink. First, the skeleton of the network is the directed graph itself which is defined as $G = (V, E)$. V is the set of all vertices (nodes) in the graph, and E represents the set of edges. Next, each edge has a finite positive edge capacity c_e . Think of the edges as a pipe, where its diameter controls how much water is allowed to pass through in a single instance. In this simile, wider pipes can pump out more water, therefore they have larger capacities, and thinner pipes can only fit so much water in one instance, so their capacity may be much lower than the larger pipes. Next, each flow network has a source node s . Source s is the start of the flow network, and it works by filling the capacities of any directed edges from itself to adjacent nodes. The source is not allowed to have any directed edges flowing into it. Finally, the sink t is the finish line of the network. The goal of a flow network is to maximize the amount of flow from s to t while abiding by Conservation of flow. Conservation of flow's constraint states that the total flow going into a node must be equal to the total flow exiting the node (excluding s and t).

3 Ford-Fulkerson and Edmonds-Karp

Ford-Fulkerson is a common augmenting-path approach for solving maximum flow networks. The algorithm is represented in two graphs: the flow network graph, and the residual graph. The flow network graph is the original graph consisting of nodes, edges with capacities, a source, and a sink. The residual graph is a derived graph that shows how much extra flow can still be pushed through each edge. In the residual graph, each edge may have a forward edge, which represents the remaining capacity after subtracting the current flow, and a backward edge, which shows how much flow can be undone/rerouted.

Ford-Fulkerson starts with zero flow in the network graph, then repeatedly searches for a path from source to sink in the residual graph, which is called an augmenting path. Once a path is found, the algorithm pushes the smallest edge capacity (the bottleneck) along that path. Then the residual graph is updated by decreasing forward capacities and increasing backward capacities accordingly.

If a better path shows up later, the residual graph allows the algorithm to reroute past decisions by sending flow backward along edges. This process repeats until no $s - t$ paths exist in the residual graph. At that point, the algorithm terminates and the current flow is guaranteed to be a maximum flow.

It is worth noting that Ford-Fulkerson is not always guaranteed to terminate if edge capacities are

irrational since it depends on the sequence of paths chosen, but this experiment's implementation is coded with integer-capacities, which will terminate and finds the optimal solution.

A common implementation uses depth-first search (DFS) to find augmenting paths in the residual graph where every edge has available capacity. While this works, it can lead to inefficient rerouting. The Edmonds–Karp algorithm is a specific implementation of Ford–Fulkerson that instead uses breadth-first search (BFS) to always select the shortest augmenting path in terms of number of edges.

4 Linear Programming

Linear programming is an extremely fast technique that is used to solve linear minimization and maximization problems. The crux of linear programming is the setup, where you define the problem within three parts: decision variables, objective, and constraints. If a variable is declared as a decision variable, the solver will manipulate the variable's values to minimize/maximize the objective; these are essentially the dials and knobs used to solve the problem. The objective is telling the solver what it should optimize. Finally the constraints narrow the search space of the solution by restricting specific actions. Once these are defined, you can pass these off to a linear program solver that will find the solution very efficiently. Flow networks can be expressed in this format too, which makes them solvable with linear programming. This is done by maximizing the amount of flow into sink t , where choosing the amount of flow per edge is a decision variable, and the two constraints are: limit the flow from being over the capacity of the given edge, and preserving conservation of flow. This project implements the CPLEX linear program solver as one of our algorithms. My code used DOcplex to define the optimization model, which was solved by the CPLEX solver, then returns the values back to Python.

5 Graph Generation

The graph generation is a crucial aspect when comparing the algorithms. The graph generation must take into account various issues to make sure these tests work properly. First, the graphs must have a guaranteed path from the source node to the sink node. This can be solved by either regenerating the graph until a path from source to sink is verified, or by manually forcing a path before connecting other nodes. This algorithm uses the latter method. Next, not only do we want to test the three algorithms for how quickly they can solve a max flow problem, but I want to know how sparse and dense graphs affect time as well.

This explains the density parameter when creating a graph: the float value $0 \leq \text{density} \leq 1$ determines the fraction of possible edges to add to each node. For example, a density of 0.1 means that 10% of the total possible edges will be generated. Next, we need a way to assign capacities. For our purposes, capacities are randomly assigned an integer value from 1 to 20. Once all of these requirements are checked, we have a valid graph to test the algorithms on.

To summarize, the graph generation goes as follows: First, create an array A of integers 1 to n , where n is the number of nodes in the graph. Node 0 represents the source (s) and node $n - 1$ is the sink (t); n is excluded when creating the array. Next, shuffle the contents in A to rearrange the path of nodes, then move node 0 and node $n - 1$ to the front and back of the array respectively. An example process of creating the array with five nodes could look like the following:

$$\{0, 1, 2, 3, 4\} \rightarrow \{4, 3, 0, 1, 2\} \rightarrow \{0, 3, 2, 1, 4\}$$

To guarantee a path from s to t , we can assign edges between adjacent nodes in the array. The capacities for the edges are stored in a dictionary where the key is a tuple (u, v) and the value is the capacity integer. Once we have a guaranteed path in the graph, we can randomly add edges until we meet our density threshold. We do this by randomly picking pairs of nodes and assigning them a capacity, skipping any node pair that could break the graph. Skipped pairs include self-loops ($u = v$), edges that already exist, edges coming into source s , and edges leaving t . The algorithm calculates the maximum possible edges in a graph after excluding the source and sink nodes with $\text{max} = (n - 1)(n - 2)$. Then, multiplying the result with the density parameter gives us the threshold: $\text{threshold} = \text{max} * \text{density}$. This becomes the max number of edges created in the graph. The algorithm adds new edges until this threshold is met. To create a full graph, the density parameter $d = 1$.

6 Results

Now that we have every component, we can begin the experimentation and plot the results. Once a graph was generated, it was copied such that all three algorithms would solve the same graph. The three algorithms were tested with five trials of every combination of graph sizes $N = \{10, 20, 50, 100, 250, 500, 750, 1000\}$, and the varying densities $D = \{0.1, 0.3, 0.5, 0.7, 0.9, 1\}$. Graph size, density, algorithm, trial number, maximum flow result, and solve time were all recorded into a csv file. The setup for creating the linear program from

Python’s code to CPLEX’s model can take time, and this was not recorded as part of CPLEX’s actual solve-time. Figure 1 illustrates the average solve time across all three algorithms, given the number of nodes in the graph. CPLEX on average consistently performs as well or better than the other competitors for graphs with 100 nodes or less, then the deviation begins around 250 nodes.

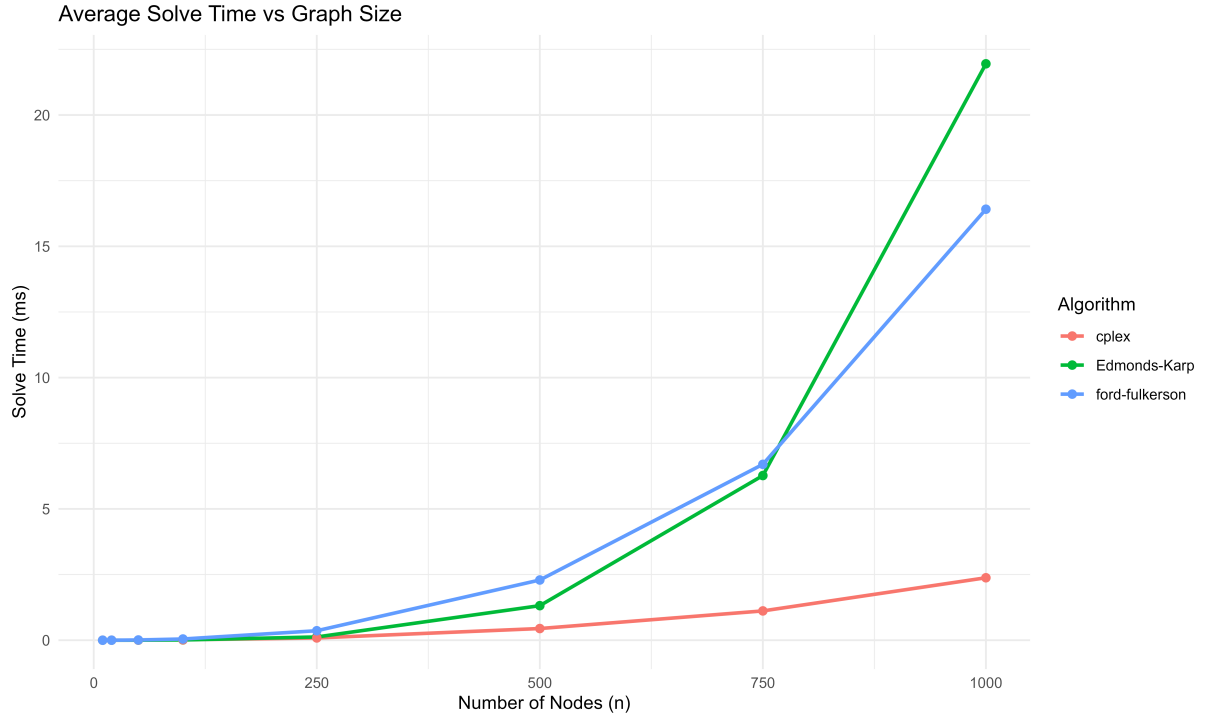


Figure 1: Line plot of solve time in milliseconds given the number of nodes in the graph, averaged across all five trials.

Interestingly enough, this raw data graph suggests that Edmonds-Karp algorithm performs minimally better for graphs with 750 nodes and less, but it loses performance as the graph’s node count increase from there. To better visualize this relationship, Figure 2 will show us how graph density affects algorithm performance, and each graph is separated by node size.

CPLEX is shown to perform worse than Ford-Fulkerson and Edmonds-Karp for the graph size of 10, but all algorithms solved these graphs practically instantly. Edmonds-Karp appears to consistently perform just as well, if not better, than CPLEX for graphs with 250 nodes or less, but this trend diminishes as graphs grow to 500-1000 nodes. After that, CPLEX consistently outperforms both Ford-Fulkerson and Edmonds-Karp by up to an average of roughly 50 milliseconds in the most dramatic case (graph size = 1000, density = 100%). Ford-Fulkerson shows a positive linear relationship between solve time and graph density as soon as a graph size of 20, while the other algorithms are mostly consistent across all densities.

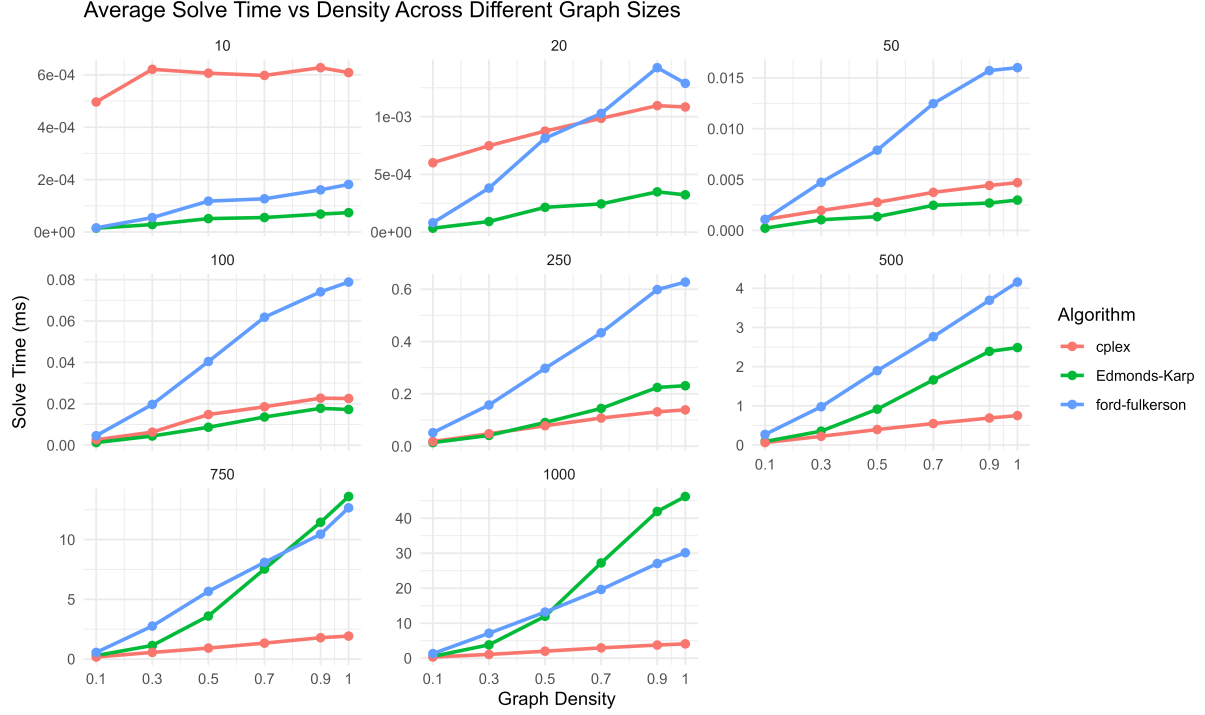


Figure 2: Line plot of the average solve time in milliseconds given the density percentage ratio of the graph, separated by graph size. A density of 0.1 means that the graph has 10% of the total possible graph density, excluding the source and sink nodes.

This linear relationship does not show for Edmonds-Karp until graphs with 500+ nodes, yet CPLEX fails to show signs of struggle regardless of node size and graph density, where the most dramatic case for CPLEX will still solve on average under 5 milliseconds (graph size = 1000, density = 100%). Despite Ford-Fulkerson consistently performing worse on average, it tended to perform better than Edmonds-Karp when there were 1000 nodes in the graph, and the densities ranged from 70%-100% coverage. This is most likely due to Edmonds-Karp's BFS technique that aims to find the shortest path, which differs from this Ford-Fulkerson's implementation that uses DFS. Both number of nodes and graph density increases the initial search space for finding the sink node because BFS checks layer-by-layer, so it makes sense why the DFS implementation would have a higher chance of finding the sink, and since 70%-100% edge density increases the chances of DFS finding the end quicker, since more paths lead to the end.

7 Conclusion

In conclusion this experiment aimed to compare Ford Fulkerson, Edmonds-Karp, and CPLEX to see how algorithm performs when solving maximum flow problems across graphs of varying sizes and densities. The results suggest that each algorithm performed well, but common trends emerged. Edmonds-Karp performed just as well, if not better, than CPLEX for smaller graphs under 250 nodes regardless of density. As graph size increased from 500 to 1000 nodes, CPLEX continued to solve each max flow graph with little effort, while Ford-Fulkerson and Edmonds-Karp's solve time began to increase linearly as the graphs became more dense. Ultimately, if Jake needed model the city of Bozeman as a flow network, Edmonds-Karp would be more than enough to identify bottlenecks in the road. On the other hand, if Jake needed to model the entire United States, the CPLEX linear programming solver is the only viable solution.